

6 Applications of Information Theory in Machine Learning

Preview of Lec. 21-24

- Information measures that appear in machine learning
- Supervised learning: logistic regression
- Unsupervised learning: Generative Adversarial Network

In the next lectures, we will talk about a few applications of information theory, especially information measures that appear in two main machine learning problems including supervised learning and unsupervised learning.

6.1 Information Measures in Supervised Learning

We now begin to uncover the role of information-theoretic measures in problems of machine learning.

6.1.1 Supervised Learning

Machine learning is about an algorithm which is defined to be a set of instructions that a computer system can execute. Formally speaking, machine learning is the study of algorithms with which one can train a computer system so that the trained system can perform a specific task of interest. There are some methodologies which help us to achieve the goal of machine learning. One specific yet very popular method is the one called: Supervised Learning. What supervised learning means is learning a function $f(\cdot)$ (indicating a functional of the machine) with the help of a supervisor.

What the supervisor means in this context is the one who provides input-output samples. Obviously the input-output samples form the data employed for training the machine, usually denoted by:

$$\{(x^{(i)}, y^{(i)})\}_{i=1}^m \quad (1)$$

where $(x^{(i)}, y^{(i)})$ indicates the i th input-output sample (or called a training sample or an example) and m denotes the number of samples. Using this notation (1), one can say that supervised learning is to:

$$\text{Estimate } f(\cdot) \text{ using the training samples } \{(x^{(i)}, y^{(i)})\}_{i=1}^m. \quad (2)$$

A common way to estimate $f(\cdot)$ is through optimization. To understand what this means, let us first explain what optimization is. Optimization is to choose an optimization variable that minimizes (or maximizes) a certain quantity of interest possibly given some constraints. There are two important quantities in the definition. One is the optimization variable which affects the interested quantity and is subject to our design. This is usually a multi-dimensional quantity. The second is the certain quantity of interest that we wish to minimize (or maximize). This is called the objective function in the field, and an one-dimensional scalar.

Now what is the objective function in the supervised learning framework? To figure this out, we need to know about the objective that supervised learning wishes to achieve. In view of the goal (2), what we want is obviously:

$$y^{(i)} \approx f(x^{(i)}), \quad \forall i \in \{1, \dots, m\}.$$

A natural question that arises is then: How to quantify closeness (reflected in the $\approx$ notation) between the two quantities: $y^{(i)}$ and $f(x^{(i)})$? One very common way that has been used in the field is to employ a function, called a loss function, usually denoted by:

$$\ell(y^{(i)}, f(x^{(i)})) \quad (3)$$

One obvious property that the loss function $\ell(\cdot, \cdot)$ should have is that it should be small when the two arguments are close, while being zero when the two are identical. Using such loss function (3), one can then formulate an optimization problem as:

$$\min_{f(\cdot)} \sum_{i=1}^m \ell(y^{(i)}, f(x^{(i)})). \quad (4)$$

Now what is optimization variable? Unfortunately, there is no variable. Instead we have a different quantity that we can optimize over: the function $f(\cdot)$, marked in red in (4). The question is then: How to introduce optimization variable? A common way employed in the field is to represent the function $f(\cdot)$ with parameters (or called weights), denoted by w , and then consider such weights as an optimization variable. Taking this approach, one can then translate the problem (4) into:

$$\min_w \sum_{i=1}^m \ell(y^{(i)}, f_w(x^{(i)})). \quad (5)$$

where $f_w(x^{(i)})$ denotes the function $f(x^{(i)})$ parameterized by w . The above optimization problem depends on how we define the two functions: (i) $f_w(x^{(i)})$ w.r.t. w ; (ii) the loss function $\ell(\cdot, \cdot)$. In machine learning, lots of works have been done for the choice of such functions.

One popular way in choosing $f_w(\cdot)$ is to use the following function that makes a smooth transition from 0 to 1:

$$f_w(x) = \frac{1}{1 + e^{-w^\top x}} = \frac{e^{w^\top x}}{e^{w^\top x} + 1}. \quad (6)$$

Notice that $f_w(x) \approx 0$ when $w^\top x$ is very small; it then grows exponentially with an increase in $w^\top x$; later grows logarithmically; and finally saturates as 1 when $w^\top x$ is very large. Actually the function (6) is a very popular one used in statistics, called the logistic function. There is another name for the function, which is the sigmoid function. There are two good things about the logistic function. First it is differentiable. The second is that it can serve as the probability for the output in the binary classifier, e.g., $\Pr(y = 1)$ where y denotes the ground-truth label in the binary classifier. So it is interpretable.

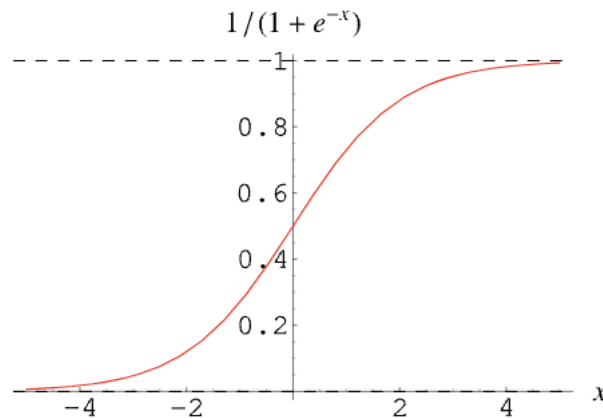


Figure 1: Sigmoid function

6.1.2 Logistic Regression

A binary classifier with the logistic function (6) is called logistic regression. See Fig. 2 for illustration.

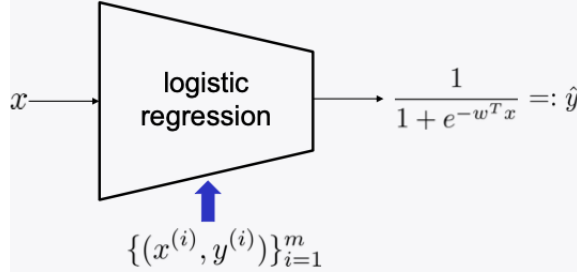


Figure 2: Logistic regression

Notice that the range of the output $\hat{y}$ of the logistic regression is in between 0 and 1: Hence, one can interpret this as a probability quantity. Now the optimality of a classifier can be defined under the following assumption inspired by such interpretation:

$$\text{Assumption: } \hat{y} = \Pr(y = 1|x). \quad (7)$$

To understand what it means in detail, consider the likelihood of the ground-truth classifier:

$$\Pr \left(\{y^{(i)}\}_{i=1}^m | \{x^{(i)}\}_{i=1}^m \right) \quad (8)$$

Notice that the classifier output $\hat{y}$ is a function of weights w . Hence, we see that assuming (7), the likelihood (8) is also a function of w . We are now ready to define the optimal w . The optimal weight, say w^* , is defined as the one that maximizes the likelihood (8):

$$w^* := \arg \max_w \Pr \left(\{y^{(i)}\}_{i=1}^m | \{x^{(i)}\}_{i=1}^m \right). \quad (9)$$

Then, an entropy-related concept appears in $\ell^*(\cdot, \cdot)$. Let us now figure this out.

Usually samples are obtained from different data $x^{(i)}$'s. Hence, it is reasonable to assume that such samples are independent with each other:

$$\{(x^{(i)}, y^{(i)})\}_{i=1}^m \quad \text{are independent over } i. \quad (10)$$

Under this assumption, we can then rewrite the likelihood (8) as:

$$\begin{aligned} \Pr \left(\{y^{(i)}\}_{i=1}^m | \{x^{(i)}\}_{i=1}^m \right) &= \frac{\Pr \left(\{y^{(i)}\}_{i=1}^m, \{x^{(i)}\}_{i=1}^m \right)}{\Pr \left(\{x^{(i)}\}_{i=1}^m \right)} \\ &= \frac{\prod_{i=1}^m \Pr \left(y^{(i)}, x^{(i)} \right)}{\prod_{i=1}^m \Pr \left(x^{(i)} \right)} \\ &= \prod_{i=1}^m \Pr \left(y^{(i)} | x^{(i)} \right). \end{aligned} \quad (11)$$

Recall the probability-interpretation-related assumption (7) made with regard to $\hat{y}$:

$$\hat{y} = \Pr(y = 1|x).$$

This implies that

$$\begin{aligned} y = 1 : \Pr(y|x) &= \hat{y} \\ y = 0 : \Pr(y|x) &= 1 - \hat{y} \end{aligned}$$

Hence, one can represent $\Pr(y|x)$ as:

$$\Pr(y|x) = \hat{y}^y (1 - \hat{y})^{1-y}.$$

Now using the notations of $(x^{(i)}, y^{(i)})$ and $y^{(i)}$, we then get:

$$\Pr(y^{(i)}|x^{(i)}) = (\hat{y}^{(i)})^{y^{(i)}} (1 - \hat{y}^{(i)})^{1-y^{(i)}}.$$

Plugging this into (11), we get:

$$\Pr\left(\{y^{(i)}\}_{i=1}^m | \{x^{(i)}\}_{i=1}^m\right) = \prod_{i=1}^m (\hat{y}^{(i)})^{y^{(i)}} (1 - \hat{y}^{(i)})^{1-y^{(i)}}. \quad (12)$$

This together with (9) yields:

$$\begin{aligned} w^* &:= \arg \max_w \prod_{i=1}^m (\hat{y}^{(i)})^{y^{(i)}} (1 - \hat{y}^{(i)})^{1-y^{(i)}} \\ &\arg \max_w \sum_{i=1}^m y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \\ &\arg \min_w \sum_{i=1}^m -y^{(i)} \log \hat{y}^{(i)} - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \end{aligned} \quad (13)$$

In fact, the term inside the summation in the last equality in (13) respects the formula of another key notion in information theory: cross entropy. In particular, in the context of a loss function, it is named the cross entropy loss:

$$\ell_{\text{CE}}(y, \hat{y}) := -y \log \hat{y} - (1 - y) \log(1 - \hat{y}). \quad (14)$$

Hence, the optimal loss function that yields the maximum likelihood solution is the cross entropy loss:

$$\ell^*(\cdot, \cdot) = \ell_{\text{CE}}(\cdot, \cdot). \quad (15)$$

Let me say a few words about why the loss function (14) is called the cross-entropy loss. Actually this comes from the definition of cross entropy. The cross entropy is defined w.r.t. two random variables. For simplicity, let us consider two binary random variables, say $X \sim \text{Bern}(p)$ and $Y \sim \text{Bern}(q)$ where $X \sim \text{Bern}(p)$ indicates a binary random variable with $p = \Pr(X = 1)$. For such two random variables, the cross entropy is defined as:

$$H(p, q) := -p \log q - (1 - p) \log(1 - q). \quad (16)$$

Notice that the formula of (16) is exactly the same as the term inside summation in (14), except for having different notations. Hence, it is called the cross entropy loss.

Then, you may now wonder why $H(p, q)$ in (16) is called cross entropy. Of course, there is a rationale. The rationale comes from the following fact:

$$\begin{aligned} H(p, q) &= -p \log q - (1 - p) \log(1 - q) \\ &= -p \log \frac{q}{p} - (1 - p) \log \frac{1 - q}{1 - p} \\ &= H_{\text{B}}(p) + D_{\text{B}}(p||q) \geq H_{\text{B}}(p) \end{aligned} \quad (17)$$

where $H_{\text{B}}(p) = -p \log p - (1 - p) \log(1 - p)$ and $D_{\text{B}}(p||q) = p \log \frac{p}{q} + (1 - p) \log \frac{1 - p}{1 - q}$. So from this, one can interpret $H(p, q)$ as an entropic-measure of discrepancy across distributions. Hence, it is called cross entropy.

The remaining question is how to solve (13). In view of (13), the optimization problem for logistic regression can be written as:

$$\min_w \sum_{i=1}^m -y^{(i)} \log \frac{1}{1 + e^{-w^\top x^{(i)}}} - (1 - y^{(i)}) \log \frac{e^{-w^\top x^{(i)}}}{1 + e^{-w^\top x^{(i)}}}. \quad (18)$$

Let $J(w)$ be the normalized version of the objective function:

$$J(w) := \frac{1}{m} \sum_{i=1}^m -y^{(i)} \log \frac{1}{1 + e^{-w^\top x^{(i)}}} - (1 - y^{(i)}) \log \frac{e^{-w^\top x^{(i)}}}{1 + e^{-w^\top x^{(i)}}}. \quad (19)$$

It turns out the above optimization belongs to convex optimization that we are familiar with. In other words, $J(w)$ is convex in optimization variable w . So one can find the optimal w by solving such convex optimization.

6.2 Information Measures in Unsupervised Learning

We face one critical challenge that arises in supervised learning. The challenge is that it is not that easy to collect labeled data in many realistic situations. In general, gathering labeled data is very expensive, as it usually requires extensive human-labour-based annotations. So people wish to do something without such labeled data. Then, a natural question that arises is: What can we do only with $\{x^{(i)}\}_{i=1}^m$? This is where the concept of unsupervised learning kicks in. Unsupervised learning is a methodology for learning something about the data $\{x^{(i)}\}_{i=1}^m$. You may then ask: What is something?

There are a few candidates for such something to learn in the field. One very popular candidate is probability distribution. In fact, this is a sort of the most complex yet most fundamental information. The probability distribution allows us to create realistic signals as we wish. The unsupervised learning method for learning such fundamental entity is: generative models. This is actually the most famous unsupervised learning method, which has received a particularly significant attention in the field nowadays. So in this course, we are going to focus on this method.

6.2.1 Unsupervised Learning: Generative Models

A generative model is defined as a mathematical model which allows us to generate fake data which has a similar distribution as that of real data. The model parameters are learned via real data so that the learned model outputs fake data that resemble real data. Here an input signal can be either an arbitrary random signal or a specifically synthesized signal that forms the skeleton of fake data. The type of the input depends on applications of interest - this will be detailed later on.

In fact, the problem of designing a generative model is one of the most important problems in statistics, so it has been a classical age-old problem in that field. This is because the major goal of the field of statistics is to figure out (or estimate) the probability distribution of data that arise in the real world (that we call real data), and the generative model plays a role as a underlying framework in achieving the goal. Actually the model can do even more - it provides a concrete function block (called the generator in the field) which can create realistic fake data. There is a very popular name in statistics that indicates such problem, that is the density estimation problem. Here the density refers to the probability distribution. Actually this problem was not that popular in the AI field until very recently, precisely 2014 when the Generative Adversarial Networks (GANs) were invented.

Now let us relate generative models to optimization. As mentioned earlier, we can feed some input signal (that we call fake input or latent signal) which one can arbitrarily synthesize. Common ways employed in the field to generate them are to use Gaussian or uniform distributions. Since it is an input signal, we may wish to use a conventional x notation. So let us use $x \in \mathbb{R}^k$ to denote a fake input where k indicates a dimension of the latent signal.

Notice that this has a conflict with real data notation $\{y^{(i)}\}_{i=1}^m$. To avoid the conflict, let us use a different notation, say $\{\hat{y}^{(i)}\}_{i=1}^m$, to denote real data. Please don't be confused with labeled data - these are not labels any more. Let $\hat{y} \in \mathbb{R}^n$ be a fake output. Considering m examples, let $\{x^{(i)}, \hat{y}^{(i)}\}_{i=1}^m$ be such fake input-output m pairs, and let $\{y^{(i)}\}_{i=1}^m$ be m real data examples.

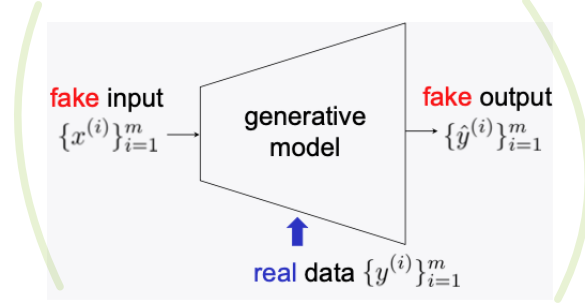


Figure 3: Generative Adversarial Networks (GANs)

Let $G(\cdot)$ be a function of the generative model. Then, the goal of the generative model can be stated as follows: Designing $G(\cdot)$ such that

$$\{\hat{y}^{(i)}\}_{i=1}^m \approx \{y^{(i)}\}_{i=1}^m \text{ in distribution.}$$

Here what does it mean by “in distribution”? To make it clear, we need to quantify closeness between two distributions. One natural yet prominent approach employed in the statistics field is to take the following two steps:

- Compute empirical distributions or estimate distributions from $\{y^{(i)}\}_{i=1}^m$ and $\{x^{(i)}, \hat{y}^{(i)}\}_{i=1}^m$. Let such distributions be

$$\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}}$$

for real and fake data, respectively.

- Next employ a well-known divergence measure in statistics which can serve to quantify the closeness of two distributions. Let $D(\cdot, \cdot)$ be one such divergence measure. Then, the similarity between $\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}}$ can be quantified as:

$$D(\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}}).$$

Taking the above natural approach, one can concretely state the goal as: Designing $G(\cdot)$ such that

$$D(\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}}) \text{ is minimized.}$$

Hence, under the approach, one can formulate an optimization problem as:

$$\min_{G(\cdot)} D(\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}}) \quad (20)$$

As you may easily notice, there are some issues in solving the above problem (21). One issue is that it is a function optimization problem which we are not familiar with. As mentioned earlier, one natural way to resolve this issue is to parameterize the function with a neural network:

$$\min_{G(\cdot) \in \mathcal{N}} D(\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}}) \quad (21)$$

There are two more yet non-trivial issues. The first is that the objective function $D(\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}})$ is a very complicated function of the knob $G(\cdot)$. Note that $\mathbb{Q}_{\hat{Y}}$ is a function of $G(\cdot)$, as $\hat{y} = G(x)$. So the objective is a twice folded composite function of $G(\cdot)$. The second is perhaps the most fundamental issue. It is not clear as to how to choose a divergence measure $D(\cdot, \cdot)$.

It turns out there are some ways to address the above issues. Very interestingly, one such way leads to an optimization problem for GANs! So we will study what that way is, and then will take the way to derive an optimization problem for GANs.

6.2.2 Generative Adversarial Networks (GANs)

Remember one challenge that we faced in the optimization problem (1): $D(\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}})$ is a complicated function of $G(\cdot)$. To address this, Ian Goodfellow, the inventor of GANs, took an indirect way to represent $D(\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}})$. He was thinking how $D(\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}})$ should behave, and his insightful observation (that we will discuss soon) led him to come up with an indirect way of mimicking the behavior. It turned out the way enabled him to explicitly compute $D(\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}})$. Below are details.

First of all, Goodfellow imagined how $D(\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}})$ should behave. What he imagined is that if one can easily discriminate real data y from fake data $\hat{y}$, then the divergence must be large; otherwise, it should be small. This naturally motivated him to:

Interpret $D(\mathbb{Q}_Y, \mathbb{Q}_{\hat{Y}})$ as a measure that can quantify the ability to discriminate.

In order to express such ability, he believed there must be something that plays a role of discriminating. So he introduced an entity that can play such role, and named it: Discriminator. Goodfellow considered a simple binary-output discriminator which takes as an input, either real data y or fake data $\hat{y}$. He then wanted to design $D(\cdot)$ such that $D(\cdot)$ well approximates the probability that the input $(\cdot)$ is real data:

$$D(\cdot) \approx \Pr((\cdot) = \text{real data}).$$

Noticing that

$$\Pr(y = \text{real data}) = 1;$$

$$\Pr(\hat{y} = \text{real data}) = 0,$$

he wanted to design $D(\cdot)$ such that

$$D(y) \text{ is as large as possible, close to } 1;$$

$$D(\hat{y}) \text{ is as large as possible, close to } 0,$$

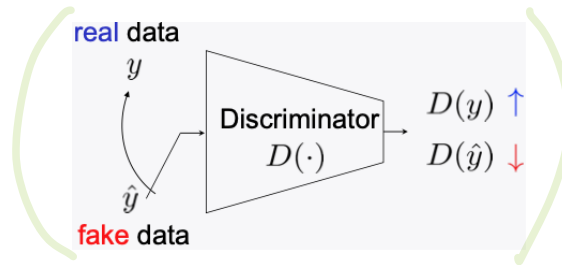


Figure 4: Discriminator

Keeping the picture in Fig. 4 in his mind, he then wanted to quantify the ability to discriminate. To this end, he first observed that if $D(\cdot)$ can easily discriminate, then we should have:

$$D(y) \uparrow; 1 - D(\hat{y}) \uparrow. \quad (22)$$

One naive way to capture the ability is simply adding the above two terms. But Goodfellow did not take the naive way. Instead he took the following logarithmic summation:

$$\log D(y) + \log(1 - D(\hat{y})) \quad (23)$$

This is also related to the binary cross entropy. Making this particular choice, the ability to discriminate for m examples can be quantified as:

$$\frac{1}{m} \sum_{i=1}^m \log D(y^{(i)}) + \log(1 - D(\hat{y}^{(i)})). \quad (24)$$

Goodfellow then imagined a two-player game in which Player 1, Discriminator $D(\cdot)$, wishes to maximize the quantified ability (24), while Player 2, Generator $G(\cdot)$, wants to minimize (24).

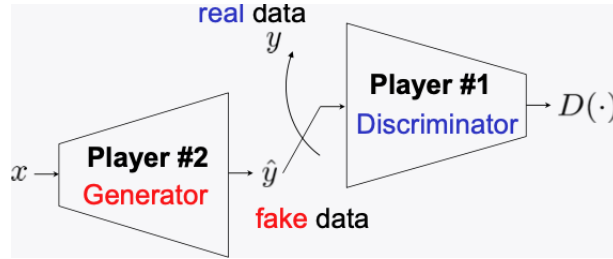


Figure 5: A two-player game

This naturally motivated him to formulate the following min max optimization problem:

$$\min_{G(\cdot)} \max_{D(\cdot)} \frac{1}{m} \sum_{i=1}^m \log D(y^{(i)}) + \log(1 - D(\hat{y}^{(i)})). \quad (25)$$

Here notice that the optimization is over the two functions of $D(\cdot)$ and $G(\cdot)$, meaning that it is still a function optimization problem. Goodfellow then parameterize the two functions with neural networks, which in turn led to the following optimization problem:

$$\min_{G(\cdot) \in \mathcal{N}} \max_{D(\cdot) \in \mathcal{N}} \frac{1}{m} \sum_{i=1}^m \log D(y^{(i)}) + \log(1 - D(\hat{y}^{(i)})). \quad (26)$$

where $\mathcal{N}$ denotes a class of neural network functions. This is exactly the optimization problem for GANs.

Remember what I mentioned earlier: the way leading to the GAN optimization is an indirect way of solving the original optimization problem:

$$\min_{G(\cdot)} D(Q_Y, Q_{\hat{Y}}) \quad (27)$$

Then, a natural question that arises is: How are the two problems (26) and (27) related? It turns out these are very much related. This is exactly where the choice of min max (instead of max min) plays the role; the other choice cannot establish a connection - it was a smart choice. It has been shown that assuming that deep neural networks can represent any arbitrary function, the GAN optimization (26) can be translated into the original optimization form (27). We will prove this below.

Let us start by simplifying the GAN optimization (26). Since we assume that $\mathcal{N}$ can represent any arbitrary function, the problem (26) becomes unconstrained:

$$\min_{G(\cdot)} \max_{D(\cdot)} \frac{1}{m} \sum_{i=1}^m \log D(y^{(i)}) + \log(1 - D(\hat{y}^{(i)})). \quad (28)$$

Notice that the objective is a function of $D(\cdot)$, and the two functions $D(\cdot)$'s appear but with different arguments: one is $y^{(i)}$, colored in blue; the other is $\hat{y}^{(i)}$, colored in red. So in the current form (28), the inner (max) optimization problem is not quite tractable to solve. In an attempt to make it tractable, let us express it in a different manner using the following notations. Define a random vector Y which takes one of the m real examples with probability $\frac{1}{m}$ (uniform distribution):

$$Y \in \{y^{(1)}, \dots, y^{(m)}\} =: \mathcal{Y}; \quad \mathbb{Q}_Y(y^{(i)}) = \frac{1}{m}, i = 1, 2, \dots, m,$$

where $\mathbb{Q}_Y$ indicates the probability distribution of Y . Similarly define $\hat{Y}$ for fake example:

$$\hat{Y} \in \{y^{(1)}, \dots, y^{(m)}\} =: \hat{\mathcal{Y}}; \quad \mathbb{Q}_{\hat{Y}}(\hat{y}^{(i)}) = \frac{1}{m}, i = 1, 2, \dots, m,$$

where $\mathbb{Q}_{\hat{Y}}$ indicates the probability distribution of $\hat{Y}$. Using these notations, one can then rewrite the problem (28) as:

$$\min_{G(\cdot)} \max_{D(\cdot)} \sum_{i=1}^m \mathbb{Q}_Y(y^{(i)}) \log D(y^{(i)}) + \mathbb{Q}_{\hat{Y}}(\hat{y}^{(i)}) \log(1 - D(\hat{y}^{(i)})). \quad (29)$$

Still we have different arguments in the two $D(\cdot)$ functions.

To address this, let us introduce another quantity. Let $z \in \mathcal{Y} \cup \hat{\mathcal{Y}}$. Newly define $\mathbb{Q}_Y$ and $\mathbb{Q}_{\hat{Y}}$ such that:

$$\mathbb{Q}_Y(z) := 0 \text{ if } z \in \hat{\mathcal{Y}} \setminus \mathcal{Y}; \quad (30)$$

$$\mathbb{Q}_{\hat{Y}}(z) := 0 \text{ if } z \in \mathcal{Y} \setminus \hat{\mathcal{Y}}. \quad (31)$$

Using the z notation, one can then rewrite the problem (29) as:

$$\min_{G(\cdot)} \max_{D(\cdot)} \sum_{z \in \mathcal{Y} \cup \hat{\mathcal{Y}}} \mathbb{Q}_Y(z) \log D(z) + \mathbb{Q}_{\hat{Y}}(z) \log(1 - D(z)). \quad (32)$$

We now see that the same arguments appear in the two $D(\cdot)$ functions.

We are ready to solve the inner optimization problem in (32). Key observations are: $\log D(z)$ is concave in $D(\cdot)$; $\log(1 - D(z))$ is concave in $D(\cdot)$; and therefore, the objective function is concave in $D(\cdot)$. This implies that the objective has the unique maximum in the function space $D(\cdot)$. Hence, one can find such maximum by searching for the one in which the derivative is zero. Taking a derivative and setting it to zero, we get:

$$\text{Derivative} = \sum_z \left[\frac{\mathbb{Q}_Y(z)}{D^*(z)} - \frac{\mathbb{Q}_{\hat{Y}}(z)}{1 - D^*(z)} \right] = 0.$$

Hence, we get:

$$D^*(z) = \frac{\mathbb{Q}_Y(z)}{\mathbb{Q}_Y(z) + \mathbb{Q}_{\hat{Y}}(z)}. \quad (33)$$

Putting this into (32), we obtain

$$\min_{G(\cdot)} \sum_{z \in \mathcal{Y} \cup \hat{\mathcal{Y}}} \mathbb{Q}_Y(z) \log \frac{\mathbb{Q}_Y(z)}{\mathbb{Q}_Y(z) + \mathbb{Q}_{\hat{Y}}(z)} + \mathbb{Q}_{\hat{Y}}(z) \log \frac{\mathbb{Q}_{\hat{Y}}(z)}{\mathbb{Q}_Y(z) + \mathbb{Q}_{\hat{Y}}(z)}. \quad (34)$$

Let us massage the objective function in (34) to express it as:

$$\begin{aligned}
& \min_{G(\cdot)} \sum_{z \in \mathcal{Y} \cup \hat{\mathcal{Y}}} \mathbb{Q}_Y(z) \log \frac{\mathbb{Q}_Y(z)}{\mathbb{Q}_Y(z) + \mathbb{Q}_{\hat{Y}}(z)} + \mathbb{Q}_{\hat{Y}}(z) \log \frac{\mathbb{Q}_{\hat{Y}}(z)}{\mathbb{Q}_Y(z) + \mathbb{Q}_{\hat{Y}}(z)} \\
& \min_{G(\cdot)} \sum_{z \in \mathcal{Y} \cup \hat{\mathcal{Y}}} \mathbb{Q}_Y(z) \log \frac{\mathbb{Q}_Y(z)}{\frac{\mathbb{Q}_Y(z) + \mathbb{Q}_{\hat{Y}}(z)}{2}} + \mathbb{Q}_{\hat{Y}}(z) \log \frac{\mathbb{Q}_{\hat{Y}}(z)}{\frac{\mathbb{Q}_Y(z) + \mathbb{Q}_{\hat{Y}}(z)}{2}} - 2 \log 2 \\
& = \min_{G(\cdot)} D_{\text{KL}}(\mathbb{Q}_Y \| (\mathbb{Q}_Y + \mathbb{Q}_{\hat{Y}})/2) + D_{\text{KL}}(\mathbb{Q}_{\hat{Y}} \| (\mathbb{Q}_Y + \mathbb{Q}_{\hat{Y}})/2) - 2 \log 2.
\end{aligned} \tag{35}$$

Slightly manipulating the above, we obtain an equivalent optimization:

$$G_{\text{GAN}}^* = \arg \min_{G(\cdot)} \frac{1}{2} \left(D_{\text{KL}}(\mathbb{Q}_Y \| (\mathbb{Q}_Y + \mathbb{Q}_{\hat{Y}})/2) + D_{\text{KL}}(\mathbb{Q}_{\hat{Y}} \| (\mathbb{Q}_Y + \mathbb{Q}_{\hat{Y}})/2) \right). \tag{36}$$

The objective is called **Jensen-Shannon divergence**. So we have formulated an optimization problem for GANs and made an interesting connection to the KL divergence.

Reference

Prof. Changho Suh's lecture note on EE623 (Lec. 15-19)